Adding Dock support to your extension

Previous: Get user input with forms

The Command Palette Dock is a persistent toolbar that stays visible at the edge of the user's screen. Extensions can provide **dock bands** — strips of items that appear in the Dock — to give users quick access to frequently used commands without opening the full Command Palette.

> ⓘ **Important**
>
> Dock support requires **Command Palette Extension SDK version 0.9 or later**. Make sure your extension project references `Microsoft.CommandPalette.Extensions` version 0.9.260303001 or higher.

Overview

Dock support is provided through two interfaces:

- **`ICommandProvider3`** — Adds the `GetDockBands()` method, which lets your extension provide dock bands.
- **`ICommandProvider4`** — Adds the `GetCommandItem(string id)` method, which enables pinning of nested commands to the Dock.

If you're using the `CommandProvider` base class from the toolkit, you can simply override these methods.

Add dock bands to your extension

To provide dock bands, override the `GetDockBands()` method in your `CommandProvider` subclass. Each `ICommandItem` returned from this method is treated as one atomic band in the Dock.

Here's a simple example that exposes a single command as a dock band:

C#

```
using Microsoft.CommandPalette.Extensions;
using Microsoft.CommandPalette.Extensions.Toolkit;

namespace <ExtensionName>;

public partial class <ExtensionName>CommandsProvider : CommandProvider
{
    private readonly ICommandItem[] _commands;
    private readonly ICommandItem _dockBand;

    public <ExtensionName>CommandsProvider()
    {
        DisplayName = "My Extension";
        Id = "com.mycompany.myextension";

        var mainPage = new <ExtensionName>Page();
        _dockBand = new CommandItem(mainPage) { Title = DisplayName };
        _commands = [new CommandItem(mainPage) { Title = DisplayName }];
    }

    public override ICommandItem[] TopLevelCommands() => _commands;

    public override ICommandItem[]? GetDockBands()
    {
        return [_dockBand];
    }
}
```

When the user enables the Dock and adds your extension's band, it appears as a button in the Dock toolbar.

Create multi-button bands with WrappedDockItem

If you want your dock band to display multiple buttons in a single strip, use the `WrappedDockItem` helper class. This lets you pass in an array of `IListItem` objects, and they're rendered as individual buttons within one band.

C#

```
using Microsoft.CommandPalette.Extensions;
using Microsoft.CommandPalette.Extensions.Toolkit;

namespace <ExtensionName>;

public partial class <ExtensionName>CommandsProvider : CommandProvider
{
```

```
    public <ExtensionName>CommandsProvider()
    {
        DisplayName = "My Extension";
        Id = "com.mycompany.myextension";
    }

    public override ICommandItem[] TopLevelCommands() => [];

    public override ICommandItem[]? GetDockBands()
    {
        var button1 = new ListItem(new OpenUrlCommand("https://github.com"))
        {
            Title = "GitHub"
        };
        var button2 = new ListItem(new OpenUrlCommand("https://learn.microsoft.com"))
        {
            Title = "Microsoft Learn"
        };

        var band = new WrappedDockItem(
            [button1, button2],
            "com.mycompany.myextension.quicklinks",
            "Quick Links");

        return [band];
    }
}
```

The `WrappedDockItem` class creates a band backed by a `ListPage` that holds your items. Each item in the array is rendered as a separate button in the Dock.

> **Tip**
>
> You can also create a `WrappedDockItem` from a single `ICommand` by using the `WrappedDockItem(ICommand command, string displayTitle)` constructor.

How dock bands work

Each `ICommandItem` returned from `GetDockBands()` represents one atomic band. How the band is rendered depends on the `Command` property of the `ICommandItem`:

Expand table

Command type	Dock behavior
`IInvokableCommand`	Renders as a single button that executes the command when selected.
`IListPage`	Renders all items on the page as individual buttons in one band.
`IContentPage`	Renders as a single expandable button with a flyout.

> **Note**
>
> All `ICommandItem` objects returned from `GetDockBands()` must have a `Command` with a non-empty `Id`. Items without an ID are ignored.

Support pinning with GetCommandItem

By default, users can only pin top-level commands and dock bands to the Dock. If you want users to be able to pin **nested commands** (commands that are deeper inside your extension), override the `GetCommandItem(string id)` method:

C#

```
public override ICommandItem? GetCommandItem(string id)
{
    var allCommands = GetAllAvailableItems();
    foreach (var item in allCommands)
    {
        if (item?.Command is ICommand cmd && cmd.Id == id)
        {
            return item;
        }
    }

    return null;
}
```

This method is called by the Command Palette when it needs to resolve a pinned command by its ID. If your extension doesn't override this method, only commands returned from `GetDockBands()` and `TopLevelCommands()` can be pinned.

Real-world example

Here's how the built-in Time & Date extension provides a dock band that shows a live clock:

C#

```
public override ICommandItem[] GetDockBands()
{
    var bandItem = new NowDockBand();
    var wrappedBand = new WrappedDockItem(
        [bandItem],
        "com.microsoft.cmdpal.timedate.dockBand",
        "Time & Date");

    return [wrappedBand];
}
```

The `NowDockBand` is a `ListItem` that updates its `Title` and `Subtitle` every minute to show the current time and date. This demonstrates how dock bands can display dynamic, live-updating content.

Related content

- Command Palette Dock
- Extensibility overview
- Extension samples

Last updated on 03/17/2026